CoRTeC Deployment Guide

Calister Nnona
research@calisternnona.com
DOI: 10.5281/zenodo.22945315

Scope. This guide is for a team that wants to deploy CoRTeC. It covers the trust boundary and what crosses it, the reference implementation and where it can run, the parameters, the prompt, a checklist, the controls mapped to standards, the failure modes the implementation guards against and the cost. The method, its privacy analysis and the measurements are in the paper (CoRTeC_arxiv.pdf). The implementation is in the same repository, Calyie/cortec. It contains the packages `cortec` and `cortec_hybrid` (version 1.2.5, Apache-2.0) and the pipeline that produced the paper's CoRTeC results.

1. Architecture

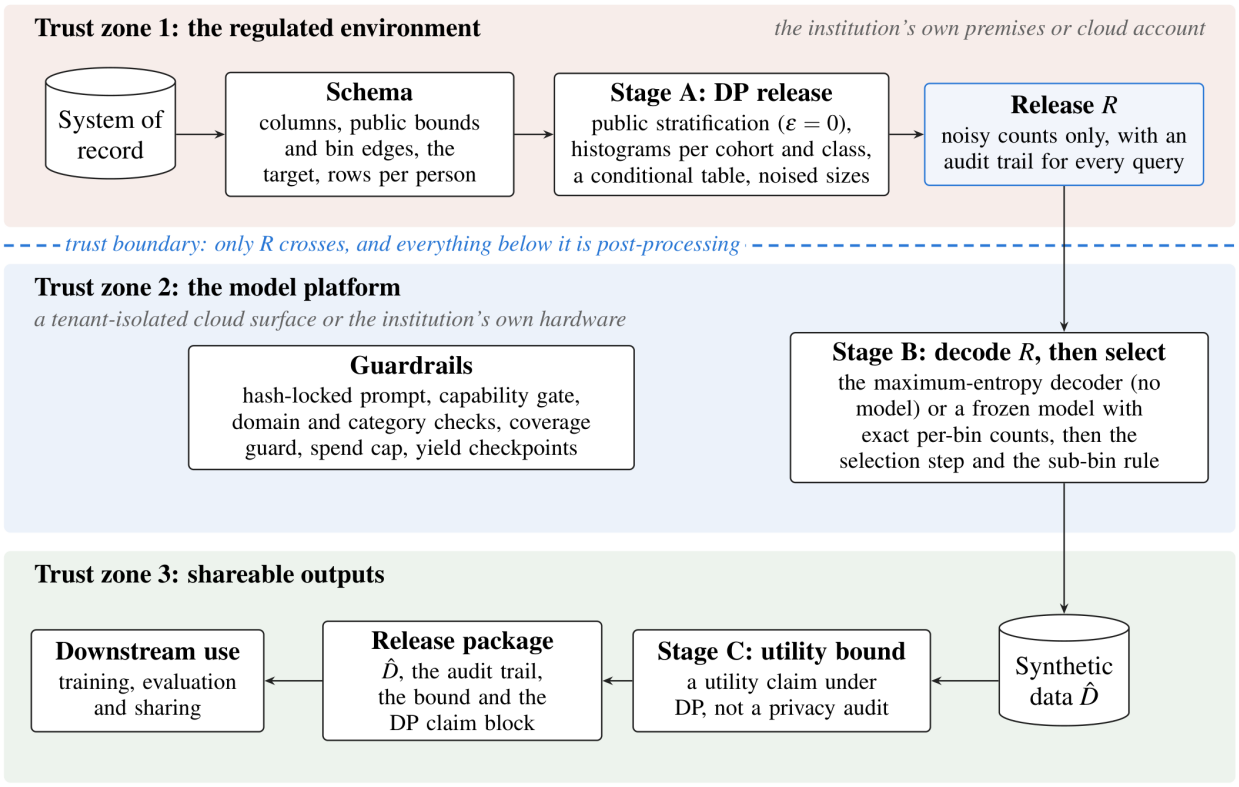


Figure 1. Trust zone 1 is the institution's regulated environment, where Stage A runs. Trust zone 2, the tenant-isolated model platform, runs Stage B on the release alone. Trust zone 3 holds the synthetic data, the Stage C bound and the release package. Only one arrow crosses the trust boundary. It carries the release R.

R is a differentially private release of counts. Stage A groups the rows into cohorts and releases four things for each cohort, all with Laplace noise: the cohort's size, its class balance, a histogram of every attribute for each outcome class and the outcome rate in each of a set of non-overlapping cells. Everything below the boundary reads only R and its own output. That includes generation, selection, the sub-bin rule and evaluation. All of it is post-processing. It can therefore be repeated as often as needed, by another team and at any later time, at no further privacy cost (paper, Theorem 1 and Appendix A).

Trust zone 2 is the enterprise surface, not a public API endpoint. We do not recommend the vendors' public developer APIs for a deployment. Instead, we recommend Claude on AWS Bedrock, GPT on Azure OpenAI or Gemini on Google Vertex AI, run in the institution's own account under its own contract. An air-gapped model on the institution's own hardware is also suitable. The privacy argument holds for any endpoint, because the request carries only R. The compliance argument is different. It rests on the contract (tenant isolation, private networking, data residency, a business associate agreement). The institution must verify it. The endpoint we measured is not the endpoint we recommend. The paper's Gemini results come from the direct public developer APIs and its Qwen3

results from a locally served model. Every capability result depends on the model weights and the prompt. The same weights are served on both surfaces. In a development measurement on one clinical dataset, we compared the two surfaces of one vendor with one release and three draws each. No metric separated them. The contractual side was never exercised by our runs.

Table 1. What the architecture provides beside the other designs.

	CoRTeC	DP-SGD fine-tuning	AIM, MST	Tab-PE
what reaches the compute	the release only, which a decoder reads	the private rows, and the trained model is the generator	the private rows, and the fitted model is the generator	the private rows, every iteration
where compute may run	anywhere	inside the regulated zone	inside the regulated zone	inside the regulated zone
a second dataset	no ε, only the generation cost	no ε, re-sample	no ε, re-sample	a new run and budget
a changed generator	no ε, re-generate	retrain	not applicable	not applicable
setup	seconds	hours on a GPU	seconds to hours	minutes

What CoRTeC offers is control over where the computation runs and what it sees. The release is a fixed workload, and its maximum-entropy decoder is Private-PGM, the same estimator that AIM uses. It gives a deployment the conditional rates it names, a file a reviewer can inspect cell by cell and a release computed in seconds. Decoded with the maximum-entropy decoder and the selection step, the auto-configured release reaches the utility of the reference AIM on Adult (paper, Sections 5.4 and 6). Fixing the workload in advance costs accuracy on pairs of columns, which an adaptive workload such as AIM's fits more closely. The marginal methods also spend their budget once and then sample for free. They suit many deployments well.

2. Deployment

```python
from cortec import Schema, Band, release_statistics, Generator

schema = Schema(
    name="encounters",
    numerical={"age": (18, 95), "length_of_stay": (1, 30)},
    categorical={"admission_type": ["Emergency", "Elective", "Urgent"],
                 "a1c_result": ["None", "Norm", ">7", ">8"]},
    target="readmitted_30d", positive="YES", negative="NO",
    bins={"age": [18, 40, 55, 70, 95], "length_of_stay": [1, 3, 6, 10, 30]},
    stratify=[("length_of_stay", [Band("los1-2", 1, 3), Band("los3-5", 3, 6),
                                  Band("los6+", 6, 31)])],
    conditional=[("admission_type",), ("admission_type", "a1c_result")],
)

release = release_statistics(schema, private_df, epsilon_total=2.0, n_min=150)
release.to_json("release.json")

gen = Generator(schema, backend="gemini", surface="vertex",
                model="gemini-3.5-flash", reasoning="on")
synthetic = gen.generate_selected(release, n_rows=5000, pool_factor=3)
print(gen.stats.calls_with_reasoning_block, "of", gen.stats.calls, "calls reasoned")
```

The schema declares only what is known without looking at the data: the column names, public bounds and bin edges, the target, the cohort rule and the conditional levels. `cortec.autoconfig.derive()` can choose the last two for you. It charges its one step that reads private data to the budget. Apart from that step, `release_statistics` is the only call that touches private data. `generate_selected` runs Stage B with the shipped defaults. On the command line, `cortec run` performs the same steps, `cortec decode` decodes with no language model and `cortec-hybrid run` runs the hybrid. The setup guide (docs/setup.md) walks through each surface, the data, the schema, Stages A to C and the evaluator.

Surfaces. `surface` is `bedrock`, `azure` or `vertex`. It changes the client class, the credential flow and the model identifier the surface expects, and nothing else. A mismatched surface or a missing setting fails before any client is created. The tests build the vendors' real client classes offline, against each vendor's public interface, and replace only the network transport. That verification used each vendor's public developer API, not its enterprise surface. One surface is verified live: Gemini 3.5 Flash on Vertex AI (an identifier under which Google now serves Gemini 3.6 Flash). It ran through a service account that holds only `roles/aiplatform.user` in the `global` location. Bedrock and Azure OpenAI are not verified live, so treat your first run on either as the live test. If a vendor refuses a call for a reason that retrying will not fix (credit, quota, authentication or permissions), the run stops at once. A local model can be reached through the `ollama` backend or any OpenAI-compatible server. The paper's Qwen3 runs used vLLM.

The hybrid. Some institutions already run AIM or MST and will not put a language model in the data path. They can keep their synthesiser, release a DP conditional table and relabel only the target column (`cortec-hybrid`). This is post-processing of two differentially private artefacts. In development measurements, it improved conditional calibration on every dataset we tried, but it moved downstream utility in both directions. The tool reports whether it helped and says when the budget is better left unspent.

3. Parameters and the release artefact

One fifth is the shipped default share for the conditional table, and `k = 3` is the shipped default pool factor. Each batch asks for what the cohort still owes after the rows already accepted.

Table 2. Shipped parameters. Reasons that do not cite the paper are development measurements.

parameter	default	reason
$\varepsilon_{\text{total}}$	2.0	the budget the paper compares architectures at: a conditional cell at the suppression floor carries its rate to within about 0.1, against about 0.2 and 0.4 at $\varepsilon = 1$ and 0.5 (paper, Section 5)
conditional share α	0.2	a fifth costs no measurable conditional accuracy once the release carries class blocks
count share γ	the package: 0.05 of the available budget plus min(0.05 of it, 0.25 of the marginal share), half to cohort sizes and half to cell supports (the paper's tables use the research pipeline's $\gamma = 0.02$, with the gate charged inside the same count query)	cohort sizes and cell supports are private counts that every $n_{\min}$ decision reads, so the package spends more on them than the paper's runs did, which is the conservative side
suppression floor $n_{\min}$	150 (refused below 50)	a rate over fewer than about 50 records is dominated by its own noise
batch size	25 rows	the largest batch on which frontier models reproduce every count line
counts per outcome class	on	when the counts were stated as totals, generators followed the class blocks unevenly
pool factor k	3 (2 on a dataset of a few thousand rows)	the selection's gain saturates between 2 and 3
pool coverage floor	1.1 times the rows each cohort owes	a pool that stopped early starved the late cohorts while every row check passed
cell-wise coverage floor	90 % of each conditioning column's released mass	between the highest failing measurement (0.858) and the lowest passing one (0.911)
sub-bin rule	`release`	the model placed values inside bins by its own prior, and the redraw changes no bin count
integer columns	from the public edges and bounds plus the majority of the pool	one "45.0" re-typed a column and delivered fractional values
reasoning	`on`	suppressing it made conditional error 3.8 times larger in a development measurement that the paper does not repeat, and the paper's open backbones ran at their default settings
Stage C tolerance τ	$\ln(k/\alpha)/(n_{\min}\,\varepsilon_{\text{cert}}) + \frac{1}{2}\max_c$	$\tilde{\rho}_c - \bar{\rho}$
Stage C minimum rows per cell	20	below it, a cell's rate can move by a third on three rows

The release artefact is a JSON document that contains no private record. For each cohort, it holds the published size, the class balance and one histogram per attribute and outcome class (or a single pooled histogram where a class fell below $n_{\min}$). For each conditional level, it holds the published positive count, size and rate of every surviving cell. An audit block records every query's ε, sensitivity, composition rule and partition key. It also records the totals: `epsilon_accounted` per row, the privacy model (pure ε, central, δ = 0, add/remove adjacency) and the privacy unit with `max_rows_per_person` and `epsilon_per_person`. A documented-gaps block lists floating-point Laplace, pretraining provenance and an undeclared privacy unit. The ledger is sealed after the release. Because the noise source ignores any seed, the stored file is the only copy. Every draw reuses it, and the generator checks its hash before each run.

4. The prompt

The system message and the cohort template are fixed and fingerprinted. A different template must be registered under a new version. Every value in every block is a released quantity. In a cohort with class blocks, the numerical and categorical blocks are given once for each outcome class.

```
SYSTEM
You generate synthetic tabular records for privacy research. You will be
given summary statistics that were computed under differential privacy.
Reproduce the distributions and relationships they describe as closely as
you can, EVEN IF THIS CONTRADICTS YOUR EXPECTATIONS about how these
variables usually relate. The statistics describe this specific
population; your general knowledge does not. Output only CSV rows. No
commentary, no explanation, no markdown fences.

USER
Generate exactly {n_rows} synthetic records for the following subgroup.

SUBGROUP: {cohort_name}

These statistics were measured on the real data for THIS subgroup, under
differential privacy. Match them. Where a statistic below conflicts with
what you would otherwise expect, follow the statistic: it is a
measurement of the actual population and your expectation is not.

NUMERIC DISTRIBUTIONS (released histograms; reproduce the shape, not just
the mean)
{numeric_block}

CATEGORICAL DISTRIBUTIONS (all released categories and their proportions)
{categorical_block}

OUTCOME RATE IN THIS SUBGROUP
{class_balance_block}

CONDITIONAL OUTCOME TABLE (P({target} = {positive} | cell), measured under
differential privacy)
This table is the most important part of this prompt. Each row states the
outcome rate for a specific cell of the population. Your generated records
must reproduce these rates, even where a rate is the opposite of what you
would expect for that group.
{conditional_block}

EXACT COUNTS FOR THIS BATCH (the number of rows, out of {n_rows}, that
must fall in each bin, each category and each outcome; these are the
counts this subgroup still owes)
{counts_block}

OUTPUT FORMAT
Exactly {n_rows} CSV data rows, plus one header row, with these columns in
this order:
{columns}

Rules:
  - every numeric value must lie within its stated range
  - {target} must be exactly "{positive}" or "{negative}"
  - do not repeat identical rows; vary records the way real data varies
  - output nothing except the header and the {n_rows} data rows
```

Two elements are essential. Removing either of them leaves no visible trace in the output. The first is the conditional table, which is the only channel for the conditional structure. The second is the instruction to follow the statistics even against expectation. Without it, the model writes its own prior. On a schema the model does not know, expect the result of the release's maximum-entropy decoder (paper, Section 4.2). Private-PGM computes it from any release at no privacy cost. The package includes this decoder as `cortec decode`. It fits the table and passes its pool through the selection step. Without any model, its single-column error is within 0.01 of the best language-model configuration's (paper, Table 3).

5. Checklist

1. **Declare the schema from public knowledge only.** That means the columns, the domain bounds, bin edges taken from clinical or regulatory convention and the target. Bands must cover the declared domain without gaps. The top band must be closed at its upper edge. Never use quantiles of the file.
2. **Set the privacy unit.** If each person has one row, $\varepsilon_{person} = \varepsilon_{row}$. Otherwise, choose one of three options. You can cap each person's contribution at their first C rows before Stage A ($\varepsilon_{person} = C\ \varepsilon_{row}$). This leaves the estimand unchanged. You can aggregate to one row per person, which changes the estimand. Or you can report $\varepsilon_{person} = k_{max}\ \varepsilon_{row}$ in place of the per-row figure. The bound tool requires `--max-rows-per-person`. The implementation refuses a declared unit whose ε_{person} exceeds 10 unless the caller acknowledges it. The acknowledgement goes into the audit trail. Capping shrinks cells, so take particular care at step 5.
3. **Choose ε and *n*.** Take the budget the regulator sets as its limit, or a tighter one. Before any generation, the release's own noise-to-signal ratio tells you whether conditioning can help. On a dataset of a few thousand rows, it will not help at a tight budget.
4. **Run Stage A once.** Store R and its audit trail as the controlled artefact and record its hash. Every draw reuses the file.
5. **Check the release before generating.** Confirm that the finest conditional table is not empty and that the noise-to-signal ratio is below 1. Note which cells cleared n_{min}. For each conditioning column, also note how much of its mass falls in bands that the surviving cells name. All of this is computed from the release at no cost to the budget.
6. **Select a reasoning-capable model on an enterprise-hosted, tenant-isolated platform or an air-gapped model on the institution's own hardware, and enable reasoning.** The vendors' public developer APIs are not a substitute. The model is the same, but the contract is not. The contract is what a compliance review assesses. Check from the reasoning block or the token count that reasoning actually took place. Treat "no count reported" as unverified. The paper's backbone ladder shows which measures improve with capability and which do not. Formatting reliability and single-column fidelity improve and level off at 14 billion parameters on Adult. Utility does not change, because it comes from the release.
7. **Decode with no model first** (`cortec decode`), then **generate** with the model where the schema is one it knows, reusing R. Draws cost no ε. The implementation refuses a pool that does not cover every released cohort at 1.1 times its share.
8. **Evaluate against floors and a ceiling.** Use a real sample at the same n, the same rows with the target shuffled and models trained on a real sample larger than n. The paper uses 4,000 rows, or the whole training split where it is smaller. Report fidelity and utility together. Compare each categorical column's full set of categories with the release. A category that is present in the release but missing from the output is a representativeness failure. No aggregate metric will report it. The generator reproduces a released conditional rate at least as sharply as the release states it, so a deployment that uses calibrated probabilities should first recalibrate them on real held-out data.
9. **Bound the result** (Stage C). Attach the bound report, the audit trail and the DP claim block to the release package.
10. **Re-verify after any change to the generator or its configuration**, including a vendor's re-routing of a model identifier.

6. Controls mapped to standards

Table 3. Each control is enforced in the implementation.

control	implementation	standard
stated DP parameters	pure ε, central, δ = 0, add/remove adjacency, privacy unit, composition rules, ε per stage and per person	NIST SP 800-226 [5]
auditable accounting	the ledger, the sole source of noise scales and sealed after release, records every query's ε, sensitivity, composition rule and partition key	NIST SP 800-226 [5]
documented gaps	floating-point Laplace [4], pretraining provenance and the privacy unit, recorded in the release audit	NIST SP 800-226 [5]
Documented utility claim in the release package	the Stage C utility transmission bound at stated confidence and ε_{cert}, which measures fitness for use, not disclosure risk	NIST SP 800-188 [6]

control	implementation	standard
re-identification risk	aggregates over cells of at least $n_{\min}$, zero exact matches in every CoRTeC configuration, four attacks validated on a positive control, the distance to the closest record and a memorisation test of the served backbone (paper, Appendix F)	ISO/IEC 27559 [2], ISO/IEC 20889 [1]
data protection by design	private data never leaves zone 1, and the prompt carries only R	GDPR [3], Article 25
expert determination route	the DP release (ε per person) and the measured re-identification evidence, **not** the Stage C bound	HIPAA Expert Determination [7]
no training on inputs	contractual, through the tenant-isolated platform	vendor terms, business associate agreement

The Expert Determination row deliberately excludes Stage C. That route requires an assessment of re-identification risk, while the bound is a utility quantity. For the same reason, the NIST SP 800-188 row describes the bound as a documented utility claim and not as a disclosure review. A utility transmission bound measures how faithfully the release reaches the synthetic rows, and it must never be presented as a privacy audit. The bound report refers only to NIST SP 800-226 and SP 800-188. The ceiling and floor that gate its verdict are scored on a real hold-out sample without noise, so keep those scores inside the institution and treat the verdict as outside the guarantee. It carries a `_standards_not_claimed` block that names the others and gives the reason. The pretraining corpus is outside the guarantee. For a method built on a proprietary pretrained model, attacking the published artefact is the one available answer to the objection that the pretraining corpus is proprietary. We claim conformance with none of the standards above.

7. Failure modes and guardrails

Each row below is a defect we ran into while building the method. Every guardrail is enforced in code.

Table 4. Failure modes and guardrails.

failure mode	what it looks like	guardrail
generator below the capability floor	plausible rows carrying none of the release's structure	a capability gate, with an explicit override required for an unmeasured model
reasoning suppressed	conditional error at the no-information floor while utility reads about 95 % of a real sample's	reasoning set explicitly and verified to have fired, with "no count" never read as zero
a second release fired silently	a fresh output directory re-spent the budget, with output indistinguishable from a correct run	a banner on every fresh release, and the artefact verified by hash
context truncation	the conditional table cut off, which looks like a model ignoring the statistics	context-fit check before the call
reasoning starves the output	empty content, no CSV	empty-content detection, and streaming above the token threshold
missing-value convention	`'None'` read as missing, and a later drop deleted 83 % of rows	`keep_default_na=False`, with category shares asserted after loading
out-of-domain rows	a credit limit of 34 against a floor of 10,000	domain-bounds rejection (public bounds, no ε)
a fraction in an integer column	one "45.0" re-typed the column, and fractional ages and hours were delivered	the integer rule
a category the schema does not declare	an integer code with a decimal point re-typed the pool	the parser normalises to declared strings and drops the rest
whole subpopulations absent	cell-wise generation emitted nothing outside the released cells: four of six racial groups at 0.000 with every dashboard green	a per-column coverage guard at 90 % of released mass, whose refusal names the column and the alternative
uniform row allocation	a 2 % cohort supplied 25 % of rows	allocation proportional to released mass
noise-dominated conditional table	conditioning cannot help, and nothing says so	release-time warning when the noise exceeds the spread of released rates
budget under-spent	the release noisier than paid for	a warning naming the shortfall, and the conditional budget split across viable levels
spend runaway	credits exhausted mid-run	a per-run cap and checkpoints on parse rate and yield, with the partial output kept
a pool that stopped early	late cohorts starved while every row check passed	pool-coverage guard at 1.1 times the rows owed
a transient rate limit	two "try later" responses abort a run	exponential backoff before a limit counts, while billing faults stay fatal

failure mode	what it looks like	guardrail
stratification bands with a gap	records at the domain maximum join no cohort	bands must tile the domain
a seeded release	byte-reproducible, reporting an ε it does not carry	a loud warning, and `guarantee_void: true` in the audit trail
an unearned per-person claim	a bound report asserting $\varepsilon_{\text{person}} = 3$ where it is 120	the privacy unit is required, and a meaningless unit must be acknowledged

The coverage guard is a heuristic mitigation, not a solved architectural component. It checks which bands of a conditioning column the released cells cover, but not the distribution inside a band they cover. How to pass a private conditional table to a language model without silent loss is still an open problem. The "exactly *k*" of a cell prompt is a request, not a guarantee. The generator compares emitted with requested positives on every call, reports the totals and warns when they differ by more than one row. It does not relabel rows. A model that matches no measured profile is marked `UNKNOWN` and is refused unless the caller overrides the check. No guardrail can tell whether the institution's data resembles the data we measured on. That is why every guardrail reports what it finds rather than only blocking.

8. Cost

At the list prices of October 2026, Gemini 3.6 Flash costs 0.75 dollars per million input tokens and 3.75 dollars per million output tokens, with reasoning billed as output. A 300-row draw at pool factor 3 takes 41 to 46 calls of 25 rows. That comes to about 1.56 dollars and half an hour. Cost and time grow linearly with the number of rows. On a locally served Qwen3-14B through vLLM, the same draw takes 42 calls and about 50 minutes on one rented NVIDIA A40. On the 4-bit 32B model, it takes about 70 minutes. The release itself takes seconds. The reference AIM fits in minutes on a CPU and then samples for free. The DP fine-tuning baselines need hours of DP-SGD on a GPU.

9. Practices and environment

The guardrail table lists the defects that affect measurement and privacy accounting. Purely infrastructural faults are omitted. Four practices caught these defects. A deployment's own evaluation should follow them. Check every metric's floor against its ceiling before trusting it. Report several student models. Inspect the raw model responses before concluding that a backend cannot do the task. Finally, make sure each check can actually fail: for example, re-release at several seeds and require every published quantity to move. In a development measurement, running the research pipeline with exact cohort counts instead of noised and charged ones moved every reported number by less than 0.0005.

Environment. The research pipeline needs pandas 2.0 or later, and scikit-learn is pinned below 1.9 for compatibility with diffprivlib. The reference AIM and MST use private-pgm with mbi 1.1 and the stable message-passing oracle. The locally served backbones run on vLLM, and DP-SGD runs on Opacus. We state these pins, because a pandas change once moved a baseline from 0.019 to 0.110 during development without any test failing. Privacy accounting is verified by instrumentation. One test patches the Laplace mechanism and asserts that exactly $\varepsilon_{\text{total}}$ is spent. The package suite also requires every noise scale drawn to be one that the ledger charged for.

Regenerating a configuration takes one call of the research driver, with the command given in the repository's README. A release already in the output directory is reused, so further draws spend nothing. The reference-AIM wrapper, the controls and the scorers are in the repository, and the draws, score files and releases are in the Zenodo record (10.5281/zenodo.22945315).

References

[1] ISO. ISO/IEC 20889:2018, Privacy enhancing data de-identification terminology and classification of techniques. 2018.

[2] ISO. ISO/IEC 27559:2022, Information security, cybersecurity and privacy protection: Privacy enhancing data de-identification framework. 2022.

[3] European Union. Regulation (EU) 2016/679 (General Data Protection Regulation), Article 25 and Recital 26. *Official Journal of the European Union*, 2016.

[4] Ilya Mironov. On significance of the least significant bits for differential privacy. In *ACM CCS*, pp. 650–661, 2012.

[5] NIST. Guidelines for evaluating differential privacy guarantees. *NIST Special Publication 800-226*, 2025.

[6] NIST. De-identifying government datasets: Techniques and governance. *NIST Special Publication 800-188*, 2023.

[7] U.S. Department of Health and Human Services. Guidance regarding methods for de-identification of protected health information in accordance with the HIPAA Privacy Rule. 2012.